

技术报告-光的力量

长上下文场景中LLM自动数据标注技术报告

摘要

1. 引言

1.1 研究背景

1.2 核心问题

1.3 方案概述

2. 提示策略设计

2.1 长上下文提示的核心挑战

2.2 结构化提示模板

2.3 提示策略的效果与局限

2.4 提示迭代过程

3. 上下文构造优化

3.1 问题定义与约束分析

3.2 示例选择策略：基于token预算的顺序填充

3.4 token预算参数选择

3.5 不同任务的示例统计

3.6 未来优化方向

4. 一致性与可扩展性

4.1 输出格式一致性：标签化解析机制

4.2 null预测的现实分析

4.3 标注标准一致性

4.4 可扩展性设计

5. 实验设计与结果分析

5.1 实验环境

5.2 数据集概览

5.3 推理参数

5.4 实验结果

5.5 逐任务深度分析

5.6 关键发现

6. 技术创新与贡献

6.2 精确的token预算管理

6.3 诚实的失败分析

7. 结论与展望

7.1 结论

7.2 局限性

8. 参考文献

附录A：代码结构

附录B：运行命令

附录C：评审问题对照

长上下文场景中LLM自动数据标注技术报告

摘要

本报告针对长上下文场景中大语言模型自动数据标注挑战，提出了一套基于In-Context Learning的完整技术方案。方案以Qwen3-4B为基座模型，通过FlagScale框架部署推理服务，围绕结构化提示模板设计、基于token预算的示例填充策略、标签化输出解析机制三个核心模块，在8个评测数据集上完成了全量标注。实验结果显示，方案在情感分类等模式明确的任务上有效预测率达90.6%，同时暴露了复杂数学推理和代码生成场景下的显著局限。本报告如实呈现方案设计、实验结果及深入的失败分析，并讨论了未能实施但具有理论价值的优化方向。

关键词：大语言模型、In-Context Learning、长上下文、数据标注、提示工程、Qwen3-4B

1. 引言

1.1 研究背景

高质量标注数据是驱动大语言模型能力跃升的核心要素。传统人工标注面临成本高、周期长、难以覆盖复杂长程语义任务等瓶颈。In-Context Learning (ICL) 通过在提示中嵌入输入-输出示例对，使模型无需参数更新即可理解任务意图并泛化至新样本，为自动化数据标注提供了新范式。

然而，超长上下文场景（数万至数十万token）给ICL带来了独特挑战：注意力稀释导致模型难以聚焦关键示例信息、海量候选示例远超上下文容量需要有效的筛选策略、输出格式在长上下文干扰下容易偏离预期。

1.2 核心问题

本方案围绕赛题提出的三个核心问题展开：

- 1. **提示策略设计**（评审权重20%）：如何设计模型指令与提示策略，在超长上下文下引导LLM稳定完成标注？
- 2. **上下文构造优化**（评审权重40%）：当候选示例远超上下文容量时，如何构造信息密集、结构合理的输入？
- 3. **一致性与可扩展性**（评审权重40%）：如何在持续标注场景中保持格式和标准的一致性？

1.3 方案概述

本方案的技术架构包含五个层次：输入层（数据集加载与任务描述提取）、示例选择层（基于token预算的顺序填充，target=8192 tokens，使用Qwen3-4B原生tokenizer精确计量）、提示构造层（角色定义+任务描述+标注指南+ICL示例+待标注文本+输出约束）、推理层（vLLM OpenAI兼容API，temperature=0.7, top_p=0.95,max_tokens=1024）、输出解析层（正则提取label标签，多标签投票，最终预测）。

2.提示策略设计

2.1 长上下文提示的核心挑战

在实验过程中，我们观察到Qwen3-4B在长上下文场景下的三个关键问题：

- 1. **格式遵从衰减**：随着输入长度增加，模型忽略label标签格式要求的概率显著上升。8个任务的null率（输出中未包含可解析label标签的比例）从9.4%到94.0%不等，平均约43.3%。
- 2. **思维链泄漏**：Qwen3-4B的thinking模式会在输出中插入think块，干扰标签解析。例如Task 4中出现thinking标签与label标签混合的输出。
- 3. **注意力远端遗忘**：当ICL示例区域占据大量token时，模型对末尾输出指令的遵从度下降。

2.2 结构化提示模板

针对上述问题，我们设计了五层结构化提示模板：

第一层：角色定义 – 建立模型的责任锚点，明确其作为专业数据标注专家的身份，强调必须严格遵循规则并将结果用label标签包裹。

第二层：任务描述 – 直接使用组委会提供的原始任务定义，不做修改，确保任务理解准确和可复现。

第三层：标注指南 – 关键设计是「鼓励推理但强制格式」：允许模型展开思维链以提升准确率，但严格要求最终答案必须用标签包裹。包含三条规则：示例学习要求、推理过程鼓励、强制输出规则。

第四层：ICL示例区 – 示例以统一格式插入，每个示例使用#号起始标记，label标签包裹输出，形成清晰的输入-输出映射关系。

第五层：输出约束重申 – 在待标注文本之后再次重申格式要求。这种首尾呼应的设计是为了对抗长上下文下的注意力衰减。

2.3 提示策略的效果与局限

效果：Task 5（情感分类）null率仅为9.4%，说明在任务模式明确、输出空间有限的场景下，提示策略能有效引导模型。Task 7（Jeopardy问答）null率30.2%，Task 2（名词动词计数）null率32.0%，表现尚可。

局限：Task 3（Collatz猜想）null率高达74.6%，Task 8（代码生成）null率94.0%，表明对需要复杂推理或代码生成的任务，提示策略无法弥补模型本身的能力短板。

2.4 提示迭代过程

我们在开发过程中迭代了三个版本的提示模板：

V1（基线版本）：简单的任务描述+示例+输入格式。问题：格式遵从率低，大量输出缺少label标签。

V2（强格式约束版本）：在V1基础上增加了严格的格式禁止示例（列举正确和错误输出），要求输出中不得包含label标签外的任何内容。问题：过度约束反而抑制了模型推理能力，在复杂任务上准确率下降。

V3（当前版本）：折中设计，鼓励内部推理但强制最终格式。在格式遵从和推理质量之间取得了更好的平衡。

3. 上下文构造优化

3.1 问题定义与约束分析

上下文构造是本方案的核心环节（评审权重40%）。问题形式化为：给定候选ICL示例集合E（N可达数百），在Qwen3-4B有效上下文窗口约束下，选择子集S使总token数不超过预算，同时最大化标注质量。

实际约束：

- Qwen3-4B原生窗口：32,768 tokens（YARN RoPE scaling可扩展至131,072）
- 提示模板固定开销：约500–800 tokens
- ICL示例可用空间：约8,000–130,000 tokens

3.2 示例选择策略：基于token预算的顺序填充

我们采用Token-Budget Sequential Packing策略：

1. 设定token预算上限 $T = 8,192$
2. 使用Qwen3-4B原生tokenizer精确计算每个示例的token数
3. 按数据集原始顺序遍历示例，逐个加入直到预算耗尽
4. token计算包含输入文本、输出文本及格式符号（约7tokens开销）

精确token计量是本策略的关键——我们不使用字符数估算或通用tokenizer，而是直接调用Qwen3-4B的AutoTokenizer，确保计算结果与模型实际消耗完全一致。

###3.3 策略选择的理由与权衡

我们在方案设计阶段系统分析了多种替代策略：

基于Embedding相似度检索：为每个测试样本动态选择语义最相关的示例。优势明显，但需要额外加载Embedding模型（如SentenceTransformer），在NPU内存已被Qwen3-4B占满90%的情况下不可行。此外，每个测试样本需独立执行检索，推理时间会增加数倍。

TF-IDF相似度检索：轻量级替代方案，不需GPU。但TF-IDF基于词频统计，对代码生成（Task 4/8）和数学推理（Task 3）等任务中的结构化输入效果有限——这些任务的关键不在于词汇相似度，而在于模式匹配和逻辑结构。

多样性采样（K-Medoids/DPP）：通过最大化示例间距离确保覆盖不同模式。需要预计算距离矩阵，且对标签分布未知的任务难以定义有意义的多样性度量。

难度分层采样：先简后难，引导渐进学习。缺乏可靠的难度度量指标——示例长度、标签类型均不能可靠代表难度。

顺序填充的合理性：组委会提供的示例集本身经过设计，前序示例涵盖了基本标注模式；在8K token预算的约束下，示例选择策略的差异远不如示例数量的影响大；确定性强、完全可复现、不引入额外随机性。

3.4 token预算参数选择

我们将默认token预算设为8,192 tokens，这一选择基于以下实验观察：

- Qwen3-4B虽然通过YARN可扩展至131K tokens，但我们观察到在超过16K tokens后，模型对中段示例的注意力显著下降，表现为输出中复述了靠前的示例模式，而忽略了中段不同模式的示例
- 8K预算通常可容纳20-50个示例（取决于单个示例长度），已足够覆盖大多数任务的主要标注模式
- 更短的输入带来更快的推理速度，单任务平均耗时约30-40分钟（8个任务串行约4.5小时）

3.5 不同任务的示例统计

各任务在8K token预算下实际使用的示例数量：

任务	候选示例数	实际选入数	单示例平均token数
Task 1 (Closest Integers)	50	~30-40	~200
Task 2 (Count Nouns/Verbs)	50	~20-30	~300
Task 3 (Collatz Conjecture)	50	~25-35	~250
Task 4 (Conala Concat)	50	~30-40	~200
Task 5 (Tweet Sadness)	50	~35-45	~180
Task 6 (MNLI Classification)	50	~15-25	~400
Task 7 (Jeopardy)	50	~20-30	~350
Task 8 (KernelGeneration)	30	~5-10	~1000+

Task8由于单个示例极长（代码片段），8K预算仅能容纳少量示例，这是其性能极差的原因之一。

3.6 未来优化方向

1. **自适应预算策略**：根据任务特点动态调整。对短示例任务可扩展至16K以包含更多示例；对长示例任务探索截断或摘要策略
2. **BM25轻量检索**：在CPU上运行，为每个测试样本选择词汇层面最相关的示例，无需GPU资源
3. **标签均衡采样**：对分类任务，确保每个标签类别有等比例的示例覆盖
4. **示例顺序优化**：利用recency bias将最相关示例放在末尾（靠近待标注文本的位置）
5. **示例压缩**：对长示例进行关键信息提取，在有限预算内塞入更多模式信息

4. 一致性与可扩展性

4.1 输出格式一致性：标签化解析机制

输出一致性是大规模标注的基础。我们采用label标签作为统一输出格式，通过正则表达式提取标注结果：

```
1 pattern = r'<label>\s*(.+?)\s*</label>'
2 content_matches = re.findall(pattern, text, re.DOTALL)
```

当模型输出包含多个label标签时（模型有时会在推理过程中多次给出答案），我们采用**多数投票机制**：统计各标签内容的出现频次，返回出现次数最多的内容。这种设计的优势在于：

1. 对模型的冗余输出具有鲁棒性
2. 当模型犹豫不决时，倾向于返回其最确信的答案
3. 解析逻辑简单，不引入额外的判断复杂度

4.2 null预测的现实分析

在实际实验中，null率（即模型输出中未包含可解析label标签的比例）是一个突出问题：

任务	有效预测数	null数	null率
Task 1 (Closest Integers)	228	272	54.4%
Task 2 (Count Nouns/Verbs)	340	160	32.0%

Task 3 (Collatz Conjecture)	127	373	74.6%
Task 4 (Conala Concat)	281	219	43.8%
Task 5 (Tweet Sadness)	453	47	9.4%
Task 6 (MNLI Classification)	324	176	35.2%
Task 7(Jeopardy)	349	151	30.2%
Task 8(Kernel Generation)	10	156	94.0%

null率与任务复杂度高度相关：简单分类任务（Task 5）null率最低，复杂推理和代码生成任务（Task 3、Task 8）null率最高。这反映了模型在处理复杂任务时，推理过程消耗了大量output token，导致无法在max_tokens限制内完成标签格式的输出。

4.3 标注标准一致性

通过以下三个层面保证标注标准的一致性：

示例一致性：所有测试样本共享同一组ICL示例。在代码实现中，示例选择仅在每个任务的第一个测试样本时执行一次，后续样本复用已选示例集。这确保了同一任务内所有样本面对完全相同的标注参考。

提示一致性：固定的角色定义、任务描述和标注指南。唯一变化的部分是待标注文本本身。

参数一致性：所有推理使用相同参数（temperature=0.7, top_p=0.95, max_tokens=1024），减少输出的随机波动。

4.4 可扩展性设计

系统采用模块化架构：

- main.py：主程序入口，负责数据加载和任务调度
- method.py：核心标注方法，包含提示构建、示例选择、输出解析
- llm_config.yaml：FlagScale/vLLM配置文件

添加新任务仅需在TASK_FILES字典中注册数据集路径，无需修改核心逻辑。推理后端通过OpenAI兼容API接口标准化，可在vLLM、FlagScale等不同后端之间无缝切换。

###4.5 多轮对话场景的讨论

当前实现：本方案采用单轮请求模式——每个测试样本独立发送一个完整的prompt（包含系统提示+用户提示），获得一次回复后提取结果。这种模式简单可靠，但未利用多轮对话的潜力。

未实现但已分析的多轮策略：

1. **渐进式标注：**先让模型分析输入文本的关键特征，再在第二轮给出标注结果。优势是推理更充分，劣势是推理时间翻倍且一致性更难保证。
2. **自校验机制：**在第一轮标注后，第二轮要求模型检查自己的答案是否符合示例中的标注标准。可能提升准确率，但同样增加推理开销。
3. **批量标注与交叉校验：**在一次对话中标注多个样本，利用上下文中已标注结果的一致性来引导后续标注。潜在风险是错误累积——一个早期错误可能影响后续所有标注。

未采用这些策略的核心原因是：在算力和时间约束下，单轮模式已经是最高效的选择，且多轮策略的效果提升未经验证。

5. 实验设计与结果分析

5.1 实验环境

硬件环境：

- GPU: Huawei Ascend 910C x 2卡
- CPU: 20核心
- 内存: 每卡64GB HBM

软件环境：

–操作系统: Linux 5.10.0–216.0.0.115.oe2203sp4.aarch64

- Python: 3.11.13
- 框架: FlagScale + vLLM Ascend 0.13.0rc1
- 模型: Qwen3–4B

模型配置：

- RoPE Scaling: YARN, factor=4.0

- 最大上下文长度: 131,072 tokens
- Tensor Parallelism: 1 (单卡部署)
- GPU内存利用率: 90%

5.2 数据集概览

数据集ID	任务名称	样本数	任务类型
1	Closest Integers	500	数值推理
2	Count Nouns and Verbs	500	语言学标注
3	Collatz Conjecture	500	数学推理
4	ConalaConcat Strings	500	字符串拼接
5	Tweet Sadness Detection	500	情感分类
6	MNLI Same Genre	500	自然语言推理
7	Jeopardy Answer Generation	500	问答生成
8	Kernel Generation	166	代码生成

5.3 推理参数

- ICL示例数量上限: 50个 (Task 1–7) , 30个 (Task 8)
- 最大输出长度: 1024 tokens
- Temperature: 0.7
- Top–P: 0.95
- Token预算: 8,192 tokens

5.4 实验结果

任务	有效预测/总数	有效率	备注
Task 1	228/500	45.6%	数值推理, 需精确计算

Task 2	340/500	68.0%	语言学任务，模式较规律
Task 3	127/500	25.4%	数学推理，Collatz序列计算困难
Task 4	281/500	56.2%	字符串拼接，部分输出混有think标签
Task 5	453/500	90.6%	二分类，任务最简单
Task6	324/500	64.8%	NLI三分类，理解能力要求高
Task 7	349/500	69.8%	开放式问答，答案多样性大
Task 8	10/166	6.0%	代码生成，逻辑一致性指标

注：上表中有效预测指成功从模型输出中解析出label标签的样本数。有效率 = 有效预测数 / 总样本数。实际准确率需要与ground truth对比，由组委会评测系统计算。

5.5 逐任务深度分析

Task 5（最佳，90.6%）：情感分类是典型的模式匹配任务，输出空间仅有"Sad"和"Not sad"两个选项。ICL示例覆盖充分，模型能快速从示例中学习到分类边界。低null率（9.4%）说明简单二分类任务下，提示策略的格式约束高度有效。

Task 7（69.8%）和Task 2（68.0%）：这两个任务表现次优。Task 7（Jeopardy问答）虽然是开放式生成，但答案通常较短（单词或短语），模型格式遵从度较好。Task 2（名词动词计数）需要精确计数，部分null可能源于模型在计数过程中消耗了过多token。

Task 1（45.6%）：Closest Integers要求从数列中找出最接近的两个整数。模型需要进行数值比较和排序，这对4B参数规模的模型有一定挑战。超过一半的null表明模型经常在数值推理后无法正确格式化输出。

Task 3（最差之一，25.4%）：Collatz猜想要求计算数列的完整迭代过程。这是纯计算任务，4B模型在多步算术运算上能力有限。74.6%的null率表明大多数情况下模型在长计算链后崩溃，无法产出结构化输出。

Task 8（最差，6.0%）：代码生成任务面临双重困难：（1）单个示例极长，8K预算仅能容纳少量示例，模型参考信息不足；（2）代码生成对语法正确性要求极高，且使用逻辑一致性指标评估（要求代码可执行且计算正确），4B模型在此任务上接近能力下限。

5.6 关键发现

- 任务复杂度是性能的主导因素：**在相同的提示策略和示例选择下，任务本身的复杂度（而非上下文长度）是决定性能的最关键变量。简单分类 > 模式匹配 > 精确计算 > 代码生成。
- null率与任务复杂度强相关：**null不仅意味着格式解析失败，更深层的原因是模型在复杂推理中耗尽了output token budget（1024 tokens），来不及输出label标签。
- 4B模型的能力边界清晰：**Qwen3-4B在模式识别和短文本分类上表现良好，但在多步数学计算和代码生成上力不从心。这一发现对ICL方案设计具有指导意义——应根据任务类型调整策略，而非一刀切。

6. 技术创新与贡献

###6.1 基于实验观察的提示工程方法论

本方案通过三个版本的提示迭代，总结出长上下文ICL场景下的提示设计原则：

–**格式约束与推理自由的平衡：**过度的格式约束会抑制模型推理能力（V2版本教训），适度鼓励推理过程同时强制最终格式（V3版本）效果最佳

- **首尾呼应策略：**在提示首部和尾部重复输出格式要求，对抗长上下文下的注意力衰减
- **角色锚定技术：**通过明确的专业角色定义，建立模型在长上下文处理中的行为基准线

6.2 精确的token预算管理

使用Qwen3-4B原生tokenizer实现精确的token计量，相比字符数估算或通用tokenizer，避免了上下文空间的浪费或溢出。这一看似简单的改进在工程实践中至关重要——不同tokenizer对相同文本的编码差异可达20–30%。

6.3 诚实的失败分析

本报告详细分析了方案在不同任务类型上的表现差异及其根本原因，包括：

- null率与任务复杂度的定量关系

- 4B模型在不同认知层次上的能力边界
- 未采用方案的理性分析与资源约束说明

这些分析对后续研究具有参考价值。

7. 结论与展望

7.1 结论

本方案基于Qwen3-4B模型和FlagScale框架，实现了长上下文场景下的ICL自动数据标注。主要贡献：

1. 提出了五层结构化提示模板，通过角色锚定和首尾呼应策略提升长上下文下的格式遵从率
2. 设计了基于token预算的顺序填充策略，使用原生tokenizer精确管理上下文空间
3. 实现了统一的标签化输出解析和多数投票机制，保证批量标注的格式一致性
4. 提供了完整的可复现代码和部署方案

7.2 局限性

1. **示例选择策略简单**：顺序填充未能充分利用示例间的语义关系，对需要特定模式匹配的测试样本可能不是最优选择
2. **缺乏多轮对话实现**：当前为单轮请求模式，未探索渐进式标注或自校验的实际效果
2. **复杂任务性能不足**：Task3（25.4%）和Task 8（6.0%）的表现反映了4B模型在数学推理和代码生成上的固有局限
3. **null率过高**：平均43.3%的null率意味着近一半的预测被浪费，优化格式遵从率是提升整体性能的最大杠杆

###7.3 未来优化方向

基于实验分析，以下方向最可能带来显著提升：

1. **降低null率**（优先级最高）：增大max_tokens至2048或4096，给模型更充裕的推理和输出空间；或在输出解析中增加回退策略，对缺少label标签的输出尝试直接提取答案
2. **任务自适应策略**：根据任务类型动态调整token预算、示例数量、输出长度等参数，而非使用统一配置
3. **轻量级示例检索**：引入BM25或简单文本匹配，在不增加GPU负担的前提下实现基本的示例-样本对齐
4. **多轮交互探索**：在时间允许的情况下，验证自校验机制对关键任务的效果

8. 参考文献

- [1] Brown T, et al. Language models are few-shot learners. NeurIPS 2020.
 - [2] Wei J, et al. Chain-of-thought prompting elicits reasoning in large language models. NeurIPS 2022.
 - [3] Liu J, et al. What makes good in-context examples for GPT-3? DeepAI 2021.
 - [4] Lu Y, et al. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. ACL 2022.
 - [5] Su J, et al. RoFormer: Enhanced transformer with rotary position embedding. arXiv:2104.09864, 2021.
 - [6] Peng B, et al. YaRN: Efficient context window extension of large language models. arXiv:2309.00071, 2023.
 - [7] Qwen Team. Qwen Technical Report. 2025.
 - [8] FlagScale Team. FlagScale: A scalable toolkit for large model training and inference. 2025.
-

附录A：代码结构

```
1  code/
2  |— main.py           # 主程序入口，任务调度
3  |— method.py        # 核心方法：提示构建、示例选择、输出解析、推理调用
4  |— api_test.py      # API连通性测试
5  |— llm_config.yaml  # FlagScale/vLLM部署配置
6  |— create_env_nvidia.sh # NVIDIA环境搭建脚本
7  |— requirements.txt  # Python依赖
8  |— README.md        # 运行说明
```

附录B：运行命令

启动vLLM服务：

```
1 cd FlagScale
2 python run.py --config-path .. --config-name llm_config action=run
```

运行单个任务：

```
1 python3 main.py --task_id 1 --tokenizer_path /root/Qwen3-4B --log_path_prefix ./outputs/
```

批量运行所有任务：

```
1 for task_id in 1 2 3 4 5 6 7 8; do
2     python3 main.py --task_id $task_id --tokenizer_path /root/Qwen3-4B --log_path_prefix ./outputs/
3 done
```

附录C：评审问题对照

评审问题	本方案响应	权重
超长上下文下的提示策略设计	五层结构化提示模板，角色锚定+首尾呼应+鼓励推理但强制格式	20%
超长上下文构造优化	基于token预算的顺序填充，原生tokenizer精确计量，系统性替代方案分析	40%
一致性与可扩展性	标签化输出+多数投票解析，示例/提示/参数三层一致性，模块化架构，多轮策略分析	40%

报告完成日期: 2026年4月6日
团队名称: 光的力量